

Option 4 design: process-aware kernel-level connection redirect

Written 2026-09-22, as the dedicated design doc for option 4 of the four network-layer alternatives ranked in `microsoft-365-copilot-interception-feasibility.md`'s Reason 1 section — the general-purpose fallback if option 3B (per-process proxy settings) doesn't apply to the M365 Copilot client. Goal: redirect a specific, brokered agent's outbound connections to this codebase's local listener, without affecting any other process on the box — the property options 1, 2, and the DNS-wildcard variant all lack.

Design, restated from the whiteboard/transcript source

1. Hook the OS's socket/connect-open event.
2. Resolve the process responsible for that event (PID → binary path → metadata).
3. Look up whether that process is an agent this codebase brokers.
4. If not, take no action — the connection proceeds untouched.
5. If it is, hand back the local listener's address instead of the real destination, for that connection only.

This reuses the process↔connection correlation this codebase's `network_egress` feature already performs for its observe/block use case — the new piece is step 5, an actual redirect action, which `network_egress` doesn't do today on either platform.

Checkpoint: verify the platform primitives before committing to this design

Three things needed independent verification before treating this option as "just new work on a platform we already understand," per the open items flagged when this option was first proposed. Checked against real platform documentation and forum-reported bug history, not assumed.

Windows: the redirect layer is real, documented, and distinct from what `network_egress` uses today

`network_egress` observes/blocks at `FWPM_LAYER_ALE_AUTH_CONNECT_V4`. The layer that actually redirects a connection's destination is a **different** one: `FWPM_LAYER_ALE_CONNECT_REDIRECT_V4`, confirmed via Microsoft's own WFP documentation:

- It exists specifically to modify the remote address and/or port of an outgoing connection, for TCP and (based on the first packet) other protocols — exactly the "return the local listener's address instead" action step 5 needs.
- Redirection is scoped to **the flow being connected**, not the whole socket — consistent with "for that connection only," not a systemic reroute.
- Available since Windows 7 — no floor-version risk on any realistically targeted fleet.
- **Callout drivers targeting this layer must register with `FwpsCalloutRegister1` or later — the older `FwpsCalloutRegister0` won't work here.** A concrete implementation detail to carry into the actual build, not just a design footnote.
- Changing the *local* address/port of a flow is a different layer (bind-redirect) and explicitly not supported at this one — irrelevant to this design, which only needs to change the remote side.

Verdict: real, well-documented, and a natural sibling to the already-proven

`ALE_AUTH_CONNECT_V4` observe/block infrastructure. New callout to write, not new platform capability to discover.

macOS: the right API is `NETTransparentProxyProvider` — a second, separate extension from the one already shipping

`network_egress`'s existing macOS piece is a `NEFilterDataProvider` system extension — built for content filtering (allow/drop), not redirection. Apple's actual redirect-capable primitive is a **different** Network Extension provider type: `NETTransparentProxyProvider`, registered via the `com.apple.networkextension.app-proxy` extension point and the `com.apple.developer.networking.networkextension` entitlement. Confirmed against Apple's own developer documentation.

This means shipping option 4 on macOS means **a second system extension alongside the existing one**, not an upgrade to it — its own registration, its own entitlement grant, its own system-extension approval step for users or MDM profiles.

The finding worth flagging before this goes further: a real, historical compatibility bug between the two extension types, specific to WebSocket upgrades

This is the one that mattered most to check, since the traffic option 4 exists to redirect **is** WebSocket traffic (the SignalR Chathub connection from Reason 2). Found via Apple Developer Forums bug reports, not speculation:

- **What broke:** when a `NEFilterDataProvider` and a `NETTransparentProxyProvider` / `NEAppProxyProvider` were both active and both matched the *same* connection, the TCP handshake succeeded but the **WebSocket upgrade specifically failed** — `SSLHandshake failed (-9810)`, and underlying socket writes failing with a broken-pipe error. Affected Safari WebSocket connections, SSH-by-hostname, `NSSStream` / `CFStream` -

based apps, Outlook's Exchange sync, and Adobe Cloud Sync — a real, broad failure mode, not an edge case.

- **When it happened:** reported starting macOS 11.1 (Big Sur). Apple tracked it as FB8918126/FB8925105.
- **Resolution:** an Apple engineer confirmed successful `wss://` connections with both extensions active after updating to **macOS 11.2 Beta 2** — i.e., **fixed**, not an open, unresolved platform bug.
- **What this means for this design:** on any realistically targeted fleet in 2026 (multiple major macOS releases past 11.2), this specific failure mode should not reproduce. It's not a blocker. It **is** a documented precedent that these two extension types interacting on the same connection is a real, previously-triggered failure class, specifically on WebSocket upgrades — exactly the traffic this design cares about most. Worth two concrete follow-ups before shipping, not just noting and moving on: 1. **Pin a real integration test:** both extensions active, a live `wss://` connection through the redirected path, on the actual minimum macOS version this product supports — not just "should be fixed," verified. 2. **Avoid the two extensions matching the same connection where possible**, as a defense-in-depth measure independent of whether the 11.2 fix fully holds: scope `NEFilterDataProvider`'s rules to exclude whatever this design redirects, so the two providers are never racing to classify the identical flow, matching the interim port-exclusion workaround Apple's own forum thread recommended before the real fix landed.

Open items before implementation

- Confirm the minimum macOS version this product actually ships on is ≥ 11.2 (near-certain, but worth stating as a checked fact, not an assumption, given how directly this bug's fix version bears on this design).
- Confirm whether `network_egress`'s existing `NEFilterDataProvider` rules could ever overlap with the destinations this design redirects (`substrate.office.com` , `copilot.cloud.microsoft` , etc.) on a real device, and scope accordingly per the defense-in-depth note above.
- On Windows, confirm `network_egress`'s existing callout registration version (already `FwpsCalloutRegister1` +, per its own design, but worth a direct check rather than an inference) so the new redirect callout is consistent with it.
- Process↔connection attribution reuse: confirm the exact function/module in `network_egress` this design should call into rather than reimplement.

Sources

- [NETransparentProxyProvider — Apple Developer Documentation](#)

- [Network Extensions Entitlement — Apple Developer Documentation](#)
- [Apple Developer Forums thread on NEFilterDataProvider/NETransparentProxyProvider WebSocket failures — FB8918126/FB8925105, fixed in macOS 11.2 Beta 2](#)
- [Filtering layer identifiers \(Fwpmu.h\) — Microsoft Learn](#)
- [ALE Layers — Microsoft Learn](#)
- [Using Bind or Connect Redirection — Microsoft Learn](#)
- `AGENTS.md` 's `network_egress` section (this repo) — the existing observe/block infrastructure this design extends